

LOBEES

AI Automation Dashboard Challenge

Reto técnico de prácticas – Nivel Junior

Duración: 5 días · 6 horas/día · 30 horas totales

Participantes: DAM / DAW

Entornos: [Claude Code](#) · [ChatGPT](#) · [Antigravity](#)

1. Introducción

1. Introducción

Bienvenido al Lobees AI Automation Dashboard Challenge.

En este reto construirás un sistema de automatización completo que integra tres tecnologías de IA: Claude Code, ChatGPT y Antigravity. El objetivo es que cada una de estas herramientas ejecute de forma autónoma un flujo de automatización sobre el CRM de Lobees y genere como resultado un dashboard HTML publicado en un servidor y linkado directamente desde la tarea en el CRM.

🎯 **Objetivo principal:** construir el mismo flujo con tres herramientas de IA distintas y comparar qué herramienta lo resuelve con menos intervención humana.

📦 **Output de cada automatización:** un archivo HTML con dashboard de métricas, publicado en servidor y accesible desde el CRM de Lobees con un clic.

⚡ **El agente de IA debe ser capaz de completar el flujo completo solo, usando el MCP de Lobees como interfaz con el CRM.**

Este reto evalúa:

- Tu capacidad de configurar y usar agentes de IA como herramientas reales de trabajo
- Tu comprensión de los Model Context Protocol (MCP) y las skills de agente
- Tu habilidad para integrar sistemas: CRM, agente IA, servidor de despliegue
- La calidad del output generado y la documentación del proceso
- Tu criterio para comparar herramientas y argumentar cuál funciona mejor y por qué

2. Conceptos Clave

2. Conceptos Clave

¿Qué es un MCP (Model Context Protocol)?

Un MCP es un servidor que expone herramientas al agente de IA. En lugar de que tú hagas una llamada a la API de Lobees manualmente, el agente de IA puede hacerlo directamente invocando una herramienta del MCP.

Sin MCP: alumno → escribe código → llama a la API → procesa respuesta → actualiza CRM

Con MCP: alumno → le dice al agente qué hacer → el agente llama al MCP → el MCP actúa sobre el CRM

El MCP es el puente entre la IA y Lobees. Tú no lo construyes: te lo damos instalado y configurado.

¿Qué es una Skill?

Una skill es un conjunto de instrucciones reutilizables que le dices al agente antes de empezar. Define exactamente qué pasos debe seguir cuando se le pide una tarea concreta. Equivale a escribir un procedimiento de trabajo que el agente ejecuta de forma consistente.

Ejemplo: la skill 'generar-dashboard' le indica al agente:

1. Obtener los datos de la tarea en Lobees
2. Construir un HTML con los KPIs
3. Publicarlo en el servidor
4. Guardar la URL en el CRM

Una vez definida la skill, el agente la ejecuta con un único prompt del usuario.

Herramientas que usarás

Cada equipo trabaja con los tres entornos siguientes para luego comparar resultados:

Herramienta	Cómo usa el MCP	Dónde se definen las skills
Claude Code	MCP nativo en .claude/mcp.json	CLAUDE.md en la raíz del proyecto
ChatGPT	Custom Actions con esquema OpenAPI	System prompt del GPT personalizado
Antigravity	Herramienta externa via HTTP wrapper	Workflow steps con instrucciones de agente

3. El MCP de Lobees

3. El MCP de Lobees

El MCP de Lobees es el servidor que os proporcionamos ya construido. Su función es exponer las operaciones del CRM como herramientas que el agente de IA puede invocar directamente, sin que vosotros tengáis que escribir código de integración.

Instalación

Ejecuta los siguientes comandos en tu terminal:

```
git clone https://github.com/mpforall/lobees-mcp
cd lobees-mcp
npm install
npm run build
```

Configuración de credenciales

Crea un archivo .env en la raíz del proyecto con las credenciales que te proporcionará el instructor:

```
LOBEES_API_URL=https://app.lobees.com/api
LOBEES_API_TOKEN=<tu_token_aquí>
DEPLOY_API_URL=https://dashboards.lobees.com/api/upload
DEPLOY_BASE_URL=https://dashboards.lobees.com
DEPLOY_TOKEN=<tu_deploy_token_aquí>
```

Herramientas disponibles del MCP

Una vez instalado, el agente tiene acceso a las siguientes 10 herramientas:

Tool	Descripción
lobees_list_pending_tasks	Lista todas las tareas de automatización pendientes en el CRM
lobees_get_task	Devuelve el detalle completo de una tarea por su ID
lobees_update_task_status	Cambia el estado de una tarea: pending / in_progress / completed / error
lobees_get_leads_by_task	Devuelve todos los leads asociados a una tarea de automatización
lobees_get_lead	Devuelve el perfil completo de un lead por su ID
lobees_write_log	Escribe una entrada en el log de actividad de la tarea, visible en el CRM
lobees_get_logs	Lee el historial de logs de una tarea

lobees_publish_dashboard	Publica el HTML del dashboard en el servidor y guarda la URL pública en el task del CRM
lobees_get_dashboard_url	Devuelve la URL del dashboard ya publicado para una tarea
lobees_update_metrics	Actualiza los contadores de progreso de la tarea en el CRM

Configuración por entorno

Claude Code

Crea o edita el archivo `.claude/mcp.json` en tu proyecto:

```
{
  "mcpServers": {
    "lobees": {
      "command": "node",
      "args": ["/ruta/absoluta/lobees-mcp/dist/index.js"],
      "env": {
        "LOBEES_API_URL": "https://app.lobees.com/api",
        "LOBEES_API_TOKEN": "TU_TOKEN",
        "DEPLOY_API_URL": "https://dashboards.lobees.com/api/upload",
        "DEPLOY_BASE_URL": "https://dashboards.lobees.com",
        "DEPLOY_TOKEN": "TU_DEPLOY_TOKEN"
      }
    }
  }
}
```

ChatGPT (GPT personalizado)

Accede a <https://chat.openai.com/gpts/editor>. En la sección 'Actions', importa el esquema OpenAPI que encontrarás en el repositorio en `/openapi.yaml`. Configura la autenticación con tu `LOBEES_API_TOKEN` como Bearer token.

Antigravity

En tu flujo de Antigravity, añade una herramienta externa apuntando al endpoint HTTP del MCP wrapper (incluido en el repositorio en `/antigravity-adapter/`). Usa la URL base que te proporcionará el instructor.

4. Skills del Agente

4. Skills del Agente

Las skills son instrucciones que defines antes de ejecutar la automatización. Le indican al agente exactamente qué pasos seguir cuando se le pide una tarea. Están diseñadas para que el agente pueda completar el flujo de principio a fin con un único prompt.

 Importante: las skills NO son código. Son instrucciones en lenguaje natural que se incluyen en el contexto del agente antes de empezar. El agente las lee y las sigue de forma autónoma.

Skill 1: generate-automation-dashboard

Esta es la skill central del reto. Define el flujo completo que el agente debe ejecutar para una tarea de automatización.

Skill: generate-automation-dashboard

Trigger: "genera el dashboard para la tarea [task_id]"

Pasos que ejecuta el agente:

1. Llama a lobees_get_task con el task_id proporcionado para obtener los datos de la tarea
2. Llama a lobees_get_leads_by_task para obtener la lista de leads asociados
3. Llama a lobees_get_logs para recuperar el historial de actividad
4. Construye un HTML autocontenido con: título de la tarea, fecha de generación, tabla de métricas (leads totales, completados por paso, confirmados, pendientes, errores), tabla de leads con su estado actual, y timeline de logs de actividad
5. El HTML debe ser responsive, usar colores corporativos (#1A56A0) y funcionar sin dependencias externas
6. Llama a lobees_publish_dashboard con el taskId y el HTML generado
7. Escribe un log con lobees_write_log: 'Dashboard generado y publicado', level: success
8. Devuelve la URL pública del dashboard al usuario

Skill 2: run-automation-task

Skill para ejecutar el flujo completo de una tarea de automatización desde cero.

Skill: run-automation-task

Trigger: "ejecuta la tarea de automatización [task_id]"

Pasos que ejecuta el agente:

1. Llama a lobees_update_task_status con status: 'in_progress'
2. Llama a lobees_get_leads_by_task para obtener los leads
3. Para cada lead: llama a lobees_write_log con 'Procesando lead [nombre]', level: info
4. Procesa cada lead según las instrucciones específicas de la tarea
5. Actualiza las métricas con lobees_update_metrics después de procesar cada lead
6. Al terminar todos los leads: llama a generate-automation-dashboard para generar el dashboard de resultados

7. Llama a `lobees_update_task_status` con `status: 'completed'`
8. Si ocurre algún error en cualquier paso: llama a `lobees_write_log` con `level: error` y luego a `lobees_update_task_status` con `status: 'error'`

Skill 3: check-pending-and-run

Skill de supervisión: busca tareas pendientes y lanza la automatización automáticamente.



Skill: check-pending-and-run

Trigger: "revisa si hay tareas pendientes y procésalas"

Pasos que ejecuta el agente:

1. Llama a `lobees_list_pending_tasks`
2. Si no hay tareas pendientes: informa al usuario y termina
3. Si hay tareas pendientes: muestra la lista al usuario con id, nombre y número de leads
4. Pregunta al usuario si debe procesar todas o solo una específica
5. Para cada tarea a procesar: ejecuta la skill `run-automation-task`
6. Al finalizar: muestra un resumen con el número de tareas procesadas y las URLs de dashboards generados

Cómo cargar las skills en cada entorno

Claude Code — archivo CLAUDE.md

Crea un archivo `CLAUDE.md` en la raíz de tu proyecto. Este archivo se carga automáticamente en el contexto del agente al iniciar Claude Code. Incluye las skills definidas arriba copiándolas en el archivo con el formato:

```
# Lobees Automation Skills

## Skill: generate-automation-dashboard
Cuando se te pida generar un dashboard para una tarea, ejecuta estos pasos:
1. lobees_get_task(taskId)
2. lobees_get_leads_by_task(taskId)
... [continúa con los pasos definidos arriba]
```

ChatGPT — System prompt del GPT

En el editor de tu GPT personalizado, pega las skills en el campo 'Instructions'. El formato es el mismo que en `CLAUDE.md`. Asegúrate de incluir también el contexto de qué es Lobees y para qué sirve cada herramienta del MCP.

Antigravity — Workflow steps

En cada workflow step de Antigravity que represente una skill, escribe las instrucciones del agente en el campo de descripción del step. Cada step equivale a un bloque de la skill.

5. Flujo Completo del Sistema

5. Flujo Completo del Sistema

Este es el flujo que tu agente de IA debe ejecutar de principio a fin para completar el reto:

PASO 1 → Usuario en Lobeas CRM crea una tarea de automatización con leads asignados
PASO 2 → El agente detecta la tarea con `lobees_list_pending_tasks`
PASO 3 → El agente marca la tarea como 'in_progress' con `lobees_update_task_status`
PASO 4 → El agente obtiene los leads con `lobees_get_leads_by_task`
PASO 5 → El agente procesa cada lead y escribe logs con `lobees_write_log`
PASO 6 → El agente actualiza métricas en tiempo real con `lobees_update_metrics`
PASO 7 → El agente genera el HTML del dashboard con los resultados
PASO 8 → El agente publica el dashboard con `lobees_publish_dashboard`
→ El HTML queda accesible en una URL pública
→ La URL queda guardada en el task del CRM automáticamente
PASO 9 → El agente marca la tarea como 'completed'
PASO 10 → El usuario abre Lobeas, ve la tarea completada y hace clic en el link del dashboard

Estructura del dashboard HTML

El HTML que genera el agente debe incluir obligatoriamente:

- Cabecera con nombre de la tarea, fecha de generación y herramienta de IA utilizada
- Tabla de métricas: leads totales, procesados, confirmados, pendientes, errores
- Tabla de leads con columnas: nombre, email, estado actual, último log
- Timeline de actividad con los logs de la tarea
- Sección de metadatos: tiempo total de ejecución, número de llamadas al MCP realizadas



El HTML debe ser autocontenido (sin dependencias externas tipo CDN).
CSS embebido, sin frameworks. Debe funcionar offline abriendo el archivo directamente.
Colores corporativos: azul primario #1A56A0, fondo claro #EBF2FB.

6. Entregables y Evaluación

6. Entregables

Repositorio Git

Cada equipo crea un repositorio con la siguiente estructura:

```
/
├── CLAUDE.md           ← Skills para Claude Code
├── .claude/
│   └── mcp.json        ← Configuración MCP para Claude Code
├── chatgpt/
│   ├── system-prompt.md ← Skills para ChatGPT
│   └── openapi-actions.yaml ← Schema de Custom Actions
├── antigravity/
│   └── workflow.json    ← Configuración del workflow
├── dashboards/         ← HTMLs generados durante el reto
│   ├── claude-code/
│   ├── chatgpt/
│   └── antigravity/
├── docs/
│   └── comparativa.md   ← Análisis comparativo de las tres herramientas
└── README.md
```

Demostración final

Al terminar el reto, cada equipo hace una demostración en vivo que debe mostrar:

- 1. Creación de una tarea en Lobees CRM con leads asignados
- 2. El agente detectando la tarea (con cualquiera de los tres entornos)
- 3. Ejecución autónoma del flujo completo en tiempo real
- 4. El dashboard HTML publicado y accesible desde la URL
- 5. El link al dashboard visible dentro del task en Lobees CRM
- 6. Presentación de la comparativa entre los tres entornos

Criterios de evaluación

Criterio	Peso	Puntos
El agente completa el flujo completo sin intervención manual	30%	30
El dashboard HTML es correcto, completo y está publicado	20%	20
La URL del dashboard aparece linkada en el task del CRM de Lobees	15%	15
Los logs de actividad son claros y visibles en el CRM	10%	10
La comparativa entre los tres entornos es argumentada y con datos	15%	15

Documentación del repositorio (README, estructura, instrucciones)	10%	10
---	-----	----

7. La Pregunta Clave

7. La Pregunta Clave del Reto

Al final del reto, la pregunta que debes poder responder con datos reales es:

¿Qué herramienta de IA completó el flujo con menos intervenciones manuales, mejor calidad de output y mayor claridad de proceso?

Para responderla, tu comparativa debe incluir para cada herramienta:

- Número de prompts necesarios para completar el flujo
- Número de errores encontrados y cómo los gestionó el agente
- Calidad del HTML generado (diseño, completitud de datos, legibilidad)
- Facilidad de configuración del MCP y las skills
- Tiempo total de ejecución
- Tu valoración subjetiva como desarrollador que lo ha usado



El equipo que mejor argumente la comparativa, con datos y criterio técnico, tiene más peso en la evaluación final que el que simplemente haga funcionar el flujo.

8. Recursos y Soporte

8. Recursos y Soporte

Recursos proporcionados por el instructor

- Credenciales de acceso a app.lobees.com (usuario y contraseña)
- Token de API de Lobees (LOBEES_API_TOKEN)
- Token de despliegue de dashboards (DEPLOY_TOKEN)
- Acceso al repositorio del MCP en GitHub
- Acceso al workspace de Antigravity del equipo

Documentación de referencia

- API de Lobees: <https://docs.lobees.com>
- Repositorio del MCP: <https://github.com/mpforall/lobees-mcp>
- Documentación MCP (Anthropic): <https://modelcontextprotocol.io>
- Claude Code docs: <https://docs.anthropic.com/claude-code>

Plataforma de colaboración

Toda la comunicación durante el reto se hace en Lobees:

- Preguntas técnicas: canal del reto en app.lobees.com
- Reporte de bloqueos: task de soporte asignada a tu equipo
- Compartir avances: publicaciones en el grupo del reto

Consejos para no bloquearte

7. Lee la documentación de la API de Lobees antes de empezar a configurar el MCP
8. Verifica que las herramientas del MCP responden antes de escribir las skills: pídele al agente que ejecute `lobees_list_pending_tasks` y confirma que devuelve datos
9. Si el agente no sigue las skills, revisa que el archivo `CLAUDE.md` (o el system prompt) está cargado correctamente
10. Si una herramienta del MCP falla, el agente debe escribir un log de error y continuar con los demás leads
11. Usa los logs del MCP en la consola para depurar problemas de autenticación



Consejo final: el reto no evalúa que todo funcione a la perfección.

Evalúa que entiendas por qué algo no funciona y cómo lo has resuelto.

Un agente que falla pero deja logs claros es mejor que uno que falla en silencio.

¡Buena suerte!

